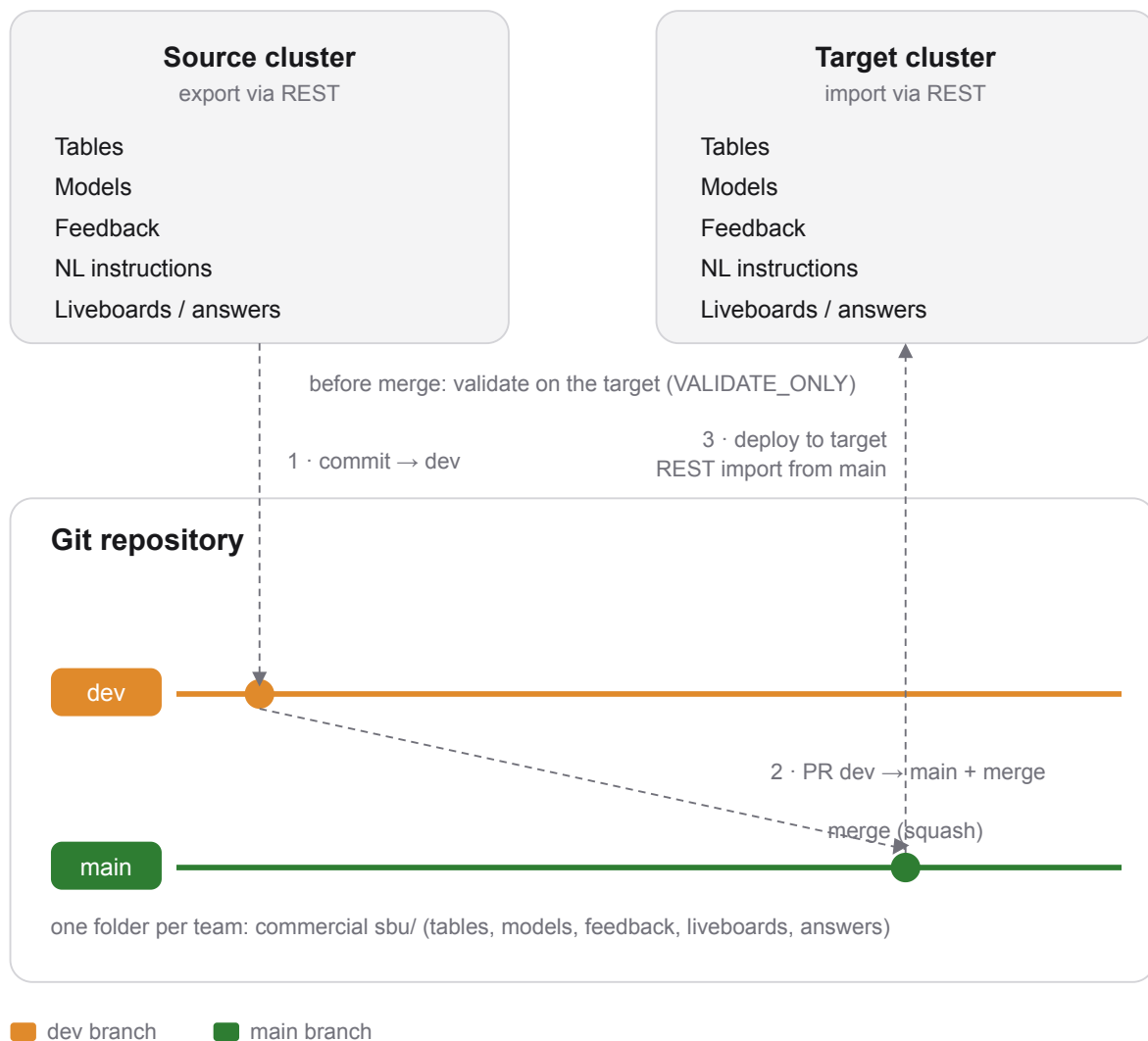


ThoughtSpot cross-cluster promotion

Git operations overview

This tool copies ThoughtSpot content (tables, models, answers, liveboards, Spotter feedback, and Spotter instructions) from one cluster or org to another. It uses a GitHub repository in the middle: every promotion is written there first (staged on `dev`, reviewed via a pull request, recorded on `main`) before it lands on the target. The tool reads from the source and writes to the target over the REST API; it does not rely on ThoughtSpot's built-in Git integration to deploy.

Git operations — source cluster to target cluster via the Git repository



Source cluster to target cluster, staged and recorded through the Git repository.

Identity: `obj_id`

Every object carries an `obj_id`, a stable name that stays the same across clusters. On import, ThoughtSpot matches by `obj_id` first: if the target already has an object with that `obj_id` it is updated in place, otherwise a new one is created. This is what keeps promotions from making duplicates.

The steps

- 1. Select what to promote.** Pick the models, answers, and liveboards on the source. The tool works out their dependencies and shows what will be included, and what would be dropped if a table is not on the target.

2. **Set and align obj_ids.** The tool checks each object has an `obj_id` that matches its counterpart on the target, suggests one where missing, and applies it on your click. This is what makes an update land in place instead of creating a copy.
3. **Export and validate.** The tool exports from the source, remaps the data layer (connection, database, schema) to the target warehouse, commits the files to the `dev` branch under the team folder, opens a `dev` to `main` pull request, and runs a dry-run validation against the target (a `VALIDATE_ONLY` import) to catch problems before anything is committed for real.
4. **Merge and import.** The pull request is squash-merged into `main` (the record of what was promoted), then the tool imports from `main` into the target over REST, in order: tables and models, then feedback, then NL instructions, then liveboards and answers.
5. **Reconcile and report.** The tool re-reads the target and confirms, per object, whether it was updated in place, newly created, a duplicate, or missing.

Order and dependencies

Order matters. Feedback and NL instructions attach to a model, so they are applied only after that model exists on the target. Liveboards and answers come last because they reference the models.

Quick glossary

- **Source / target cluster** - where content comes from and goes to; different clusters or orgs.
- **dev branch** - disposable staging branch; every export is written here first.
- **main branch** - the reviewed, merged record of what has been promoted.
- **Pull request** - the review step that proposes moving `dev` into `main`.
- **obj_id** - the stable cross-cluster identity that decides update-in-place versus create-new.
- **VALIDATE_ONLY** - a dry-run import that checks for errors without changing anything.